

# Graph Traversal: Keys and Rooms

Authored by: Arpit Raj, LNMIIT Jaipur

Date: July 30, 2026

## BFS Solution (Queue)

- Treat each room as a node and each key as a directed edge.
- Start traversal from Room 0.
- Mark Room 0 as visited ( $seen[0]=1$ ) and push it into the queue.
- Maintain cnt to count unique rooms visited.
- While the queue is not empty:
  - Pop the front room.
  - Visit all rooms whose keys are present in the current room.
  - If a room is unvisited:
    - Mark it visited.
    - Push it into the queue.
    - Increment cnt.
- After BFS finishes:
  - If  $cnt == n$ , every room is reachable  $\rightarrow$  return true.
  - Otherwise  $\rightarrow$  return false.



**Time Complexity:  $O(V + E)$**



**Space Complexity:  $O(V)$**

## C++ Code

```
class Solution {
public:
    bool canVisitAllRooms(vector<vector<int>>& rooms) {

        //basic idea ye he ki , 0th room se chaalu krenge
        //har ek neighbour room me ghuste jaayenge
        //by the end saare room seen he to true
        int n=rooms.size();
        vector<int> seen(n,0);
        queue<int> q;
        int cnt=0; //to store how many unique rooms visited

        q.push(0);
        seen[0]=1;
        cnt++;

        while(!q.empty()){
            int curr=q.front();
            q.pop();

            for(auto &room:rooms[curr]){
                if(!seen[room]){
                    seen[room]=1;
                    q.push(room);
                    cnt++;
                }
            }
        }

        if(cnt==n) return true;
        else return false;
    }
};
```

## DFS Solution (Recursion)

---

- Treat each room as a node and each key as a directed edge.
- Start DFS from Room 0.
- In every DFS call:
  - Mark current room as visited.
  - Increment cnt.
  - Recursively visit every unvisited neighbouring room.
- DFS automatically explores one complete path before backtracking.
- After DFS completes:
  - If  $cnt == n$ , every room was visited → return true.
  - Otherwise → return false.



**Time Complexity:  $O(V + E)$**



**Space Complexity:  $O(V)$  (visited array + recursion stack)**

## C++ Code

```
class Solution {
public:
    int cnt=0;

    void dfs(vector<vector<int>>& rooms, vector<int>& seen, int currroom){

        seen[currroom]=1;
        cnt++;

        for(auto &it:rooms[currroom]){
            if(!seen[it]){
                dfs(rooms,seen,it);
            }
        }
    }

    bool canVisitAllRooms(vector<vector<int>>& rooms) {
        int n=rooms.size();
        vector<int> seen(n,0);
        dfs(rooms,seen,0);

        //rooms , seen , start room

        if(cnt==n) return true;
        else return false;
    }
};
```